COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURES AND ALGORITHM	**SECTION:**	CPE21S1
DATE PERFORMED:	9/8/2026	**DATE SUBMITTED:**	9/9/2026
NAME:	LOREEN JAE D. PILAPIL	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 7.1 SORTING ALGORITHMS PT.1

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize=100;
int unsortedArr[maxSize];

void displayArr(int arr[]);

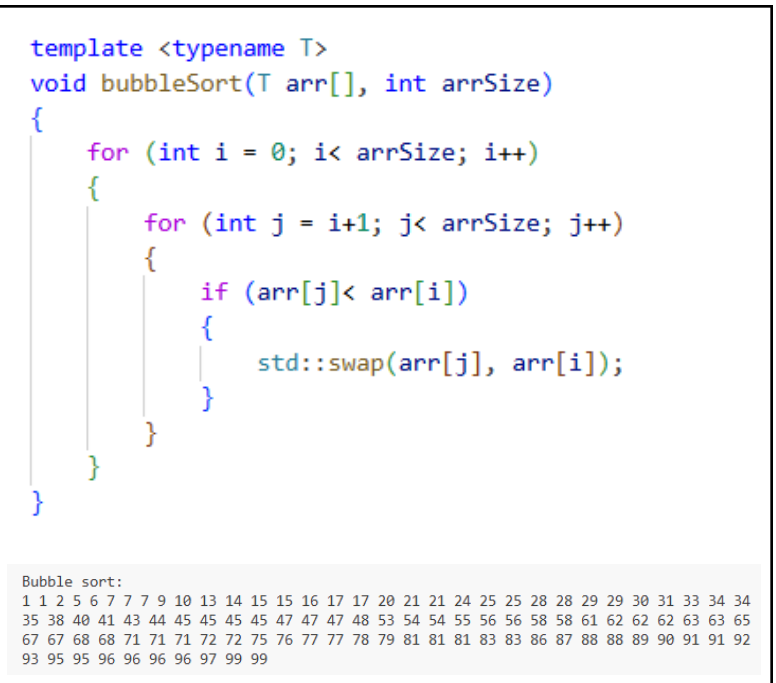
int main()
{
    srand(time(0));

    for (int i =0; i <maxSize; i++)
    {
        unsortedArr[i]=rand() % 100;
    }
```

```
Unsorted:
15 88 89 41 62 72 25 47 34 45 77 16 99 29 91 56 45 45 7 83 17 55 88 56 81 95 14 10 78 2
5 1 86 58 43 62 6 7 21 1 90 87 38 35 29 24 81 92 63 65 96 44 72 93 67 33 15 91 28 76 68
 71 5 68 47 13 95 99 30 79 2 45 96 96 61 34 20 54 48 31 47 67 75 83 9 96 28 62 63 77 7
58 40 21 17 54 53 97 71 71 81
```

Observations	I noticed that this creates an array of 100 random numbers.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	I noticed that this sorts an array of 100 random numbers using bubble sorting technique where it arranges numbers from smallest to largest by repeatedly comparing two numbers and swapping them.

```
template <typename T>
void bubbleSort(T arr[], int arrSize)
{
    for (int i = 0; i< arrSize; i++)
    {
        for (int j = i+1; j< arrSize; j++)
        {
            if (arr[j]< arr[i])
            {
                std::swap(arr[j], arr[i]);
            }
        }
    }
}
```

```
Bubble sort:
1 1 2 5 6 7 7 7 9 10 13 14 15 15 16 17 17 20 21 21 24 25 25 28 28 29 29 30 31 33 34 34
35 38 40 41 43 44 45 45 45 45 47 47 47 48 53 54 54 55 56 56 58 58 61 62 62 62 63 63 65
67 67 68 68 71 71 71 72 72 75 76 77 77 78 79 81 81 81 83 83 86 87 88 88 89 90 91 91 92
93 95 95 96 96 96 96 97 99 99
```

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	

```
template <typename T>
int routine_smallest(T A[], int maxSize, int k)
{
    int position = k;
    T smallestElem=A[k];

    for (int j = k +1; j< maxSize; j++)
    {
        if (A[j] <smallestElem)
        {
            smallestElem= A[j];
            position = j;
        }
    }

    return position;
}
template <typename T>
void selectionSort(T arr[], int arrSize)
{
    int POS, temp;

    for (int i = 0; i <arrSize; i++)
    {
        POS = routine_smallest(arr, arrSize, i);
        std::swap(arr[i], arr[POS]);
    }
}

#endif
```

```
Selection sort:
1 1 2 5 6 7 7 7 9 10 13 14 15 15 16 17 17 20 21 21 24 25 25 28 28 29 29 30 31 33 34 34
35 38 40 41 43 44 45 45 45 45 47 47 47 48 53 54 54 55 56 56 58 58 61 62 62 62 63 63 65
67 67 68 68 71 71 71 72 72 75 76 77 77 78 79 81 81 81 83 83 86 87 88 88 89 90 91 91 92
93 95 95 96 96 96 96 97 99 99
```

Observations	I noticed that it sorts an array of 100 random numbers using a selection sorting technique where it arranges numbers from smallest to largest by finding the smallest number and putting it in the correct position.

Table 7-3. Selection Sort Algorithm

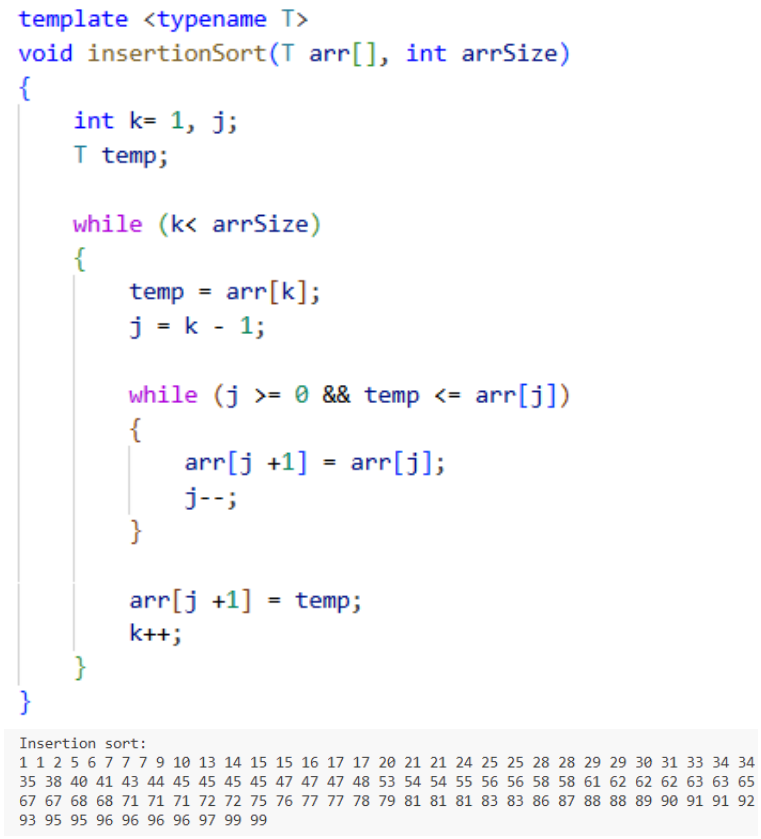
Code + Console Screenshot	
Observations	I noticed that it sorts an array of 100 random numbers using insertion sorting technique where it arranges numbers from smallest to largest by taking one number at a time and inserting it into the correct position.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0...100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H


template <typename T>
void bubbleSort(T arr[],int arrSize)
{
    for (int i =0; i< arrSize - 1; i++){
        for (int j =0; j< arrSize - i - 1; j++){
            if (arr[j] > arr[j +1]){

                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}

#endif
```

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize= 100;
int unsortedArr[maxSize];

void displayArr(int arr[])
{
    for (int i=0; i<maxSize; i++){
        std::cout<< arr[i]<< " ";
    }
}


int main()
{
    srand(time(0));
    for (int i = 0; i<maxSize; i++){
        unsortedArr[i]= rand() % 5 +1;
    }

    std::cout <<"Unsorted Votes:\n";
    displayArr(unsortedArr);

    bubbleSort(unsortedArr, maxSize);

    std::cout <<"\nSorted Votes:\n";
    displayArr(unsortedArr);

    int count[6]= {0};
    for (int i= 0; i<maxSize; i++) {
        count[unsortedArr[i]]++;
    }

    std::cout << "\nResults:\n";
    std::cout << "Candidate 1: "<< count[1]<<" votes\n";
    std::cout << "Candidate 2: "<< count[2]<<" votes\n";
    std::cout << "Candidate 3: "<< count[3]<<" votes\n";
    std::cout << "Candidate 4: "<< count[4]<<" votes\n";
    std::cout << "Candidate 5: "<< count[5]<<" votes\n";
```

```
    int winner= 1;
    for (int i= 2; i<= 5;i++){
        if (count[i] > count[winner])
        {
            winner= i;
        }
    }

    std::cout << "\nWinner: Candidate "<< winner << std::endl;

    return 0;
}
```

Output Console Showing Sorted Array	Manual Count Count	Result of Algorithm
Sorted Votes: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	Candidate 1: 25 Candidate 2: 19 Candidate 3: 19 Candidate 4: 20 Candidate 5: 17 Winner: Candidate 1 - Bo Dalton Capistrano	Results: Candidate 1: 25 votes Candidate 2: 19 votes Candidate 3: 19 votes Candidate 4: 20 votes Candidate 5: 17 votes Winner: Candidate 1
Sorted Votes: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	Candidate 1: 21 Candidate 2: 21 Candidate 3: 26 Candidate 4: 16 Candidate 5: 16 Winner: Candidate 3 - Deja Jayla Banaga	Results: Candidate 1: 21 votes Candidate 2: 21 votes Candidate 3: 26 votes Candidate 4: 16 votes Candidate 5: 16 votes Winner: Candidate 3
Sorted Votes: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	Candidate 1: 18 Candidate 2: 20 Candidate 3: 20 Candidate 4: 20 Candidate 5: 22 Winner: Candidate 5 - Franklin Relano Castro	Results: Candidate 1: 18 votes Candidate 2: 20 votes Candidate 3: 20 votes Candidate 4: 20 votes Candidate 5: 22 votes Winner: Candidate 5

Question: Was your developed vote counting algorithm effective? Why or why not?

Yes, because the manually counted votes are the same as the result of the algorithm, therefore the algorithm was accurate and effective in counting the votes. Also, I chose bubble sort because it is simple to understand and easy to implement.

Conclusion:

In this activity, I learned about three sorting algorithms, bubble sort, insertion sort, and selection sort. I then applied bubble sort to organize and count 100 randomly generated votes. The algorithm was effective because its results matched the manually counted votes, showing that it produced accurate results.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.